Subject: Full-Stack Development

Subject Code: 24CSP-210

Project Report: Job Portal

Submitted by:
Student Name: Harshit Rana
UID: 24BCS11749
Branch: BE-CSE
Section/Group: 24BCS_KRG_616-B

Submitted to:
Mr. Deep Prakash Gupta

1. PROJECT OVERVIEW

1.1 Background & Purpose

The Indian job market is highly competitive, with millions of professionals seeking opportunities across diverse sectors. It was conceived to offer a lightweight yet feature-complete portal that can be deployed quickly, serve as a foundation for enterprise expansion, or function as a standalone recruitment tool for small-to-medium businesses.

Traditional job portals often suffer from slow interfaces, poor mobile responsiveness, and cumbersome registration flows. It addresses these pain points by adopting a single-page application (SPA) architecture with protected routes, an intuitive filter system, and graceful fallback to local dummy data when the backend is unavailable.

1.2 Project Objectives

- Provide a secure, token-based authentication system for job seekers.
- Enable multi-criteria job discovery through search, filter, and category browsing.
- Deliver a professional profile management experience with skills and preferences.
- Expose a clean RESTful API for future mobile or third-party integrations.
- Maintain a monorepo structure separating frontend and backend concerns cleanly.

1.3 Scope of the System

The current version (v1.0) covers four core screens — Authentication, Profile, Job Search, and Job Detail — with in-memory storage for user accounts and eight pre-loaded job listings. The system is intentionally scoped to validate UX flows and API contracts, with a clear migration path to MongoDB or PostgreSQL for persistent storage.

2. FEATURES & FUNCTIONALITY

2.1 Secure Authentication

Users register with name, email, password, and an optional role (job seeker / recruiter). Passwords are hashed with bcryptjs (salt rounds = 10). On successful registration or login, a JSON Web Token (JWT) is issued with a 7-day expiry. The token is stored in localStorage and attached to all subsequent Axios requests via a default header. The AuthContext provider restores the session on page reload, eliminating repeated logins.

2.2 User Profile Management

The Profile page serves as the user's personal dashboard. It allows editing of professional headline, location, skills (comma-separated tag input), years of experience, expected salary range, preferred work mode (remote / on-site / hybrid), and a short bio. A profile completion indicator encourages users to fill all fields before browsing jobs.

2.3 Job Search & Multi-Criteria Filtering

The Job Search page presents all eight pre-loaded listings in a responsive card grid. A persistent filter panel exposes six filter dimensions: keyword/title/company, location, industry category, employment type, experience band, and remote-only toggle. Filters are applied both client-side (immediate feedback) and via the /api/jobs/search endpoint (server-side filtering). A reset control clears all active filters instantly.

2.4 Job Detail & Application

Each job card links to a dedicated detail page displaying full job description, required skills (rendered as badges), salary, location, employment type, posting date, and application deadline. An Apply Now button simulates the application submission. Urgent listings are highlighted with a prominent badge.

2.5 Responsive Navigation

The Navbar renders contextually: unauthenticated users see Login/Register links, while authenticated users see Profile, Jobs, and Logout. Route guards in App.js use the AuthContext to redirect unauthenticated users to /auth and logged-in users away from the auth page, preventing redundant access.

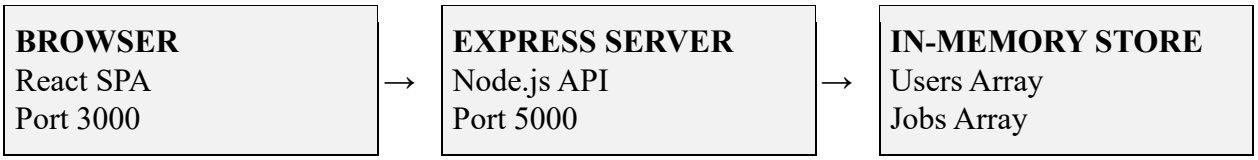
2.6 Offline Resilience & Fallback Data

If the backend server is unreachable (e.g., during development or demo without a running Node process), the JobSearchPage gracefully falls back to a built-in DUMMY_JOBS array that mirrors the backend dataset. This allows frontend-only demonstrations without any server dependency.

3: SYSTEM ARCHITECTURE

3.1 Architectural Pattern

Portal follows a classic client-server architecture with a strict separation of concerns. The frontend is a React SPA that communicates with the backend exclusively through HTTP/REST. CORS is enabled globally on the Express server, and the React development proxy forwards /api/* calls to localhost:5000.



4. APPLICATION FLOW

4.1 Authentication Flow

The following sequence describes the end-to-end user journey from first visit through authenticated access:

#	Step	Action / Description	Route
1	**Initial Visit**	Browser hits root URL; App.js redirects to /auth	/auth
2	**Register / Login**	User submits credentials; AuthContext calls /api/auth/register or /login	/auth
3	**Token Storage**	JWT stored in localStorage; Axios header updated globally	—
4	**Profile Redirect**	On success, user is automatically navigated to the Profile page	/profile
5	**Profile Completion**	User fills professional details and clicks Find Jobs	/profile
6	**Job Browse**	Job Search page loads with all 8 listings from API or fallback data	/jobs
7	**Filter & Search**	User applies keyword, location, category, type or remote filter	/jobs
8	**View Job Detail**	Clicking a card navigates to the full job description page	/jobs/:id
9	**Apply**	Apply Now button triggers application action on the detail page	/jobs/:id

10	**Logout**	Navbar Logout clears token, user state, and redirects to /auth	/auth

5: CONCLUSION

Job Portal demonstrates a well-structured, production-oriented approach to full-stack web development. By cleanly separating the React frontend from the Express backend, the project achieves a high degree of maintainability and extensibility. The JWT authentication system, multi-filter job search, and profile management collectively provide a complete recruitment experience.

The fallback data strategy ensures the application remains demonstrable and testable in any environment, while the RESTful API design allows easy integration with future mobile clients or third-party services. Replacing the in-memory user store with a persistent database (MongoDB Atlas or PostgreSQL) is the primary next step to make the platform fully production-ready.